Object Oriented Programming (CSSE7023)

Assignment 1 — Semester 2, 2025

School of Electrical Engineering and Computer Science

The University of Queensland

Due August 29th 17:00 AEST

One must learn by doing the thing; for though you think you know it, you have no certainty, until you try.
— Sophocles

Revision 1.0.0

Overview This assignment delivers practical experience developing a Java project based on a supplied specification. The software specification is provided in the form of "**Javadoc**", which describes the classes that your assignment must implement. You will be assessed on your ability to:

- implement a program that complies with the specification,
- and develop code that conforms to the style conventions of the course.

Task Object-Oriented Programming offers several advantages over Procedural Programming, particularly for complex or evolving software systems:

- Encapsulation: Object-Oriented Programming bundles the data and methods that operate on that data into each object – this reduces accidental interference – internals are hidden, protecting the state.
- Modularity: In Object-Oriented Programming, code is organized into classes, making it easier to break up a large system into manageable pieces and allows different programmers to work on different parts of the program (in this assignment, you will only write some of the classes – others will already be written).
- Inheritance: Classes can inherit from existing ones, reusing and extending their behavior which reduces duplication and simplifies maintenance – we will include some inheritance is this assignment.
- Maintainability: Changes can be made in isolated classes without affecting unrelated parts making it easier to fix bugs and update features – and to refactor the codebase for Individual Assignment 2!

In this first Individual Assignment 1, we will implement an old (1971) teletype game that Mike Mayfield wrote in a procedural language (BASIC), using object-oriented programming (Java).

Mike Mayfield wrote STTR1, based on the Star Trek© television series at the time, to work on a teletype (pure text printing on continuous paper, no graphics).

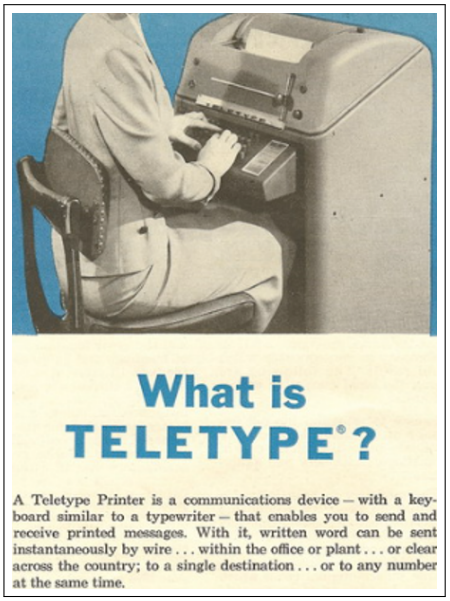

Figure 1: Part of an advertisement from the Teletype Corporation `https://medium.com/@nasthia861_74285/teletypes-and-their-legacy-cfe5e50f0760`.

We will use the 1973 public-domain version (published by HP) for HP2000 BASIC.

```
10  REM ****  HP BASIC PROGRAM LIBRARY  ******************************
20  REM
30  REM       STTR1: STAR TREK
40  REM
50  REM       36243  REV B  --  10/73
60  REM
70  REM ****  CONTRIBUTED PROGRAM  ************************************
71  REM Shortened to <=72 char lines 3/08
72  REM Converted to paper-tape basic by Terry Newton 9/07
73  REM Strings, PRINT USING and other syntax matters re-coded
74  REM Variable names changed to single letters and arrays...
...
```

Obviously, single-letter variable names, etc., do not meet our CSSE7023 style guide! So there are a number of changes.

Adjustments In converting our game from BASIC to Java, me made a few changes to keep things clear.

The top-level package name is sttrj ('j' for Java) in place of the original STTR1.

The "Quadrants"[1] (there's 64 of them in our "Galaxy"), in the original game were numbered as a 2-dimensional (2D) coordinate pair (x, y) from 1 to 8 left-to-right and from 1 to 8 top-to-bottom (BASIC indices start at 1), however we shall use default Java indices starting from 0, so the Quadrants are numbered 0-7 left to right and 0-7 top to bottom – i.e. Quadrants go from (0, 0) in the top left corner of the Galaxy to (7, 7) in the bottom right corner of the Galaxy. Remember, this is the "outside" view – how you represent your quadrants internally is up to you, but a Quadrant must have an `XyPair` between (0, 0) and (7, 7).

Yes! Mike Mayfield laid-out his Galaxy with increasing "y" values going DOWN the page –i.e. the origin (0, 0) is the top left corner, not the bottom right corner as you may expect. Why would he do this? Is this just a poor design choice by a high-school student? No! It is an appropriate choice for printing to a teletype – the first line gets printed, then the paper feeds and the next line is printed UNDER the first, then the paper feeds and next line is printed UNDER that, etc. – printing rows 1-8 (our 0-7) increasing indices go DOWN the page…

Note! The radial directions are still numbered in the same order as the increasing quadrants of the standard Cartesian plane – directions 1-8 follow the same radials as 0-315°in the standard Cartesian plane. In our game, directions are currently INTEGERS!

```
...
5880  PRINT "  'COURSE IS IN A CIRCULAR NUMERICAL         4    3    2"
5890  PRINT "  VECTOR ARRANGEMENT AS SHOWN.                `.  :  .'"
5900  PRINT "  INTERGER AND REAL VALUES MAY BE               `.:.'"
5910  PRINT "  USED.  THEREFORE COURSE 1.5 IS             5---<*>---1"
5920  PRINT "  HALF WAY BETWEEN 1 AND 2.                     .':`."
5930  PRINT "                                              .'  :  `."
5940  PRINT "  A VECTOR OF 9 IS UNDEFINED, BUT            6    7    8"
5950  PRINT "  VALUES MAY APPROACH 9."
5960  PRINT "                                                COURSE"
5970  PRINT "  ONE 'WARP FACTOR' IS THE SIZE OF"
5980  PRINT "  ONE QUADRANT.  THEREFORE TO GET FROM"
5990  PRINT "  QUADRANT 5,6 TO 5,5 YOU WOULD USE COURSE 3, WARP"
6000  PRINT "  FACTOR 1. COORDINATES ARE SPECIFIED USING X,Y NOTATION"
6002  PRINT "  WITH X 1-8 FROM LEFT-RIGHT AND Y 1-8 FROM TOP-BOTTOM."
...
```

Common Mistakes Please carefully read Appendix A. It outlines common and critical mistakes which you must avoid to prevent a loss of grades. If at any point you are even slightly unsure, please check as soon as possible with course staff.

[1] A note for those for whom English is a second language: the word "quadrant" really means 1 part of a shape divided into 4, but in this game, the term "Quadrant" refers to 1 part of our 2-dimensional "Galaxy" divided into 64 areas.

Plagiarism All work on this assignment is to be your own individual work. By submitting the assignment you are claiming it is your own work. You *may* discuss the overall general design of the application with other students. Describing details of how you implement your design with another student is considered to be **collusion** and will be counted as plagiarism.

Do **not** show other students your code. Do **not** look at other students' code. Do **not** copy fragments of code from other students. Code supplied by course staff (from *this* semester) is acceptable, but must be clearly acknowledged as described in the next paragraph. You do not need to acknowledge the provided code that was supplied with this specification – it is part of the specification and is not included in the academic integrity checks. Code generated by third-party tools is also acceptable, but **must** also be clearly acknowledged, see *Generative Artificial Intelligence* below.

You may find ideas of how to solve problems in the assignment through external resources (e.g. StackOverflow, textbooks, ...). If you use these ideas in designing your solution you ***must*** cite them. To cite a resource, provide the full bibliographic reference for the resource in file called `refs.md`. The `refs.md` file **must** be in the root folder of your project. For example:

```
> cat refs.md
[1] E. W. Dijkstra, "Go To Statement Considered Harmful," _Communications of the ACM_,
    vol 11 no. 3, pp 147-148, Mar. 1968. Accessed: Mar. 6, 2024. [Online]. Available:
    https://www.cs.utexas.edu/users/EWD/transcriptions/EWD02xx/EWD215.html
[2] B. Liskov and J. V. Guttag, _Program development in  Java: abstraction,
    specification, and object-oriented design_. Boston: Addison-Wesley, 2001.
[3] T. Hawtin, "String concatenation: concat() vs '+' operator," stackoverflow.com,
    Sep. 6, 2008. Accessed: Mar. 8, 2024. Available:
    https://stackoverflow.com/questions/47605/string-concatenation-concat-vs-operator
```

In the code where you use the idea, cite the reference in a comment. For example:

```java
/**
 * What method1 does.
 * [1] Used a method to avoid gotos in my logic.
 * [2] Algorithm based on section 6.4.
 */
public void method1() ...

/**
 * What method2 does.
 */
public void method2() {
    System.out.println("Some " + "content.") // [3] String concatenation using + operator.
```

You must be familiar with the university's policy on plagiarism.

`https://uq.mu/rl553`

If you have questions about what is acceptable, please ask course staff.

Generative Artificial Intelligence Artificial Intelligence (AI) (e.g. ChatGPT or Copilot) and Machine Translation (MT) are emerging tools that may support students in completing this assessment task. Students may ***appropriately*** use AI and/or MT in completing this assessment task. Students ***must clearly reference*** any use of AI or MT ***in each instance***.

A failure to reference generative AI or MT use may constitute student misconduct under the Student Code of Conduct.

Please note that submitting an assignment that was written by AI, correctly referenced throughout, clearly identifying that all code was written by AI, is not plagiarism, but since the code was not written by the student, it ***has no academic merit*** and so would receive zero marks.

To pass this assessment, students may be required to demonstrate detailed comprehension of their submission independent of AI and MT tools, as part of the assignment marking process.

All submitted code will be subject to electronic plagiarism and collusion detection. The assignments are to be worked on individually and must be your own work except where the use of code written or provided by other entities is explicitly permitted by the assignment specification, ***and any such code is referenced in the manner required in the assignment specification.***

This is a learning exercise and you will harm your learning if you use AI tools inappropriately. Remember, you will be required to write code, by hand, in the final exam.

Specification

This document specifies the assignment conditions and assessment. Read this assignment specification *carefully*. The software specification is provided in the form of Javadocs.

- Implement the classes ***exactly*** as described in the Javadocs.
- Read the Javadocs carefully and understand the program specification ***before*** programming.
- Do not change the public specification in ***any*** way, ***including*** changing the names of, or adding additional, public classes, interfaces, methods, or fields.
- You are encouraged to add additional ***private*** members, classes, or interfaces as you see fit.

Getting Started

To get started, read this document right to the end – do not jump to the Javadoc or start coding before you read this assignment specification – there are important assessment details following.

After you have finished this document, you can download the Javadoc specification from **BlackBoard (Assessment → Individual Assignment 1)**, unpack it and open the `index.html` in your favourite browser, or access it ready-to-view at the link below.

`https://csse7023.uqcloud.net/assessment/assign1/docs/`

Next, download the provided code from **BlackBoard**. Extract the archive into the directory you want to work from and open it with *IntelliJ*.

Task

Fully implement each of the following classes, as described in the Javadoc specification:

sttrj.game This package contains the core of the game with all the objects and their rules for interaction.

- XyPair.java (recommended starting point)
- Stat.java
- Entity.java
- Star.java
- Starbase.java
- Enterprise.java
- Klingon.java
- Quadrant.java
- Galaxy.java
- Game.java

sttrj.input The InputHandler (provided) accepts keyboard input, parses it, and invokes appropriate actions.

- InputHandler.java (will be provided)
- **sttrj.input.validator** (sub-package) A packaged set of input validators, any one of which is used by the InputHandler in validating the users input. The base Validator class (Validator.java) will be provided and accepts all input. You will write the other Validator subclasses:
  - Validator.java (will be provided)
  - EnergyValidator.java
  - CourseValidator.java
  - WarpCoordValidator.java

sttrj.screen This entire package will be provided and implements the user interface.

See the next page for a class diagram of the specification for Individual Assignment 1.

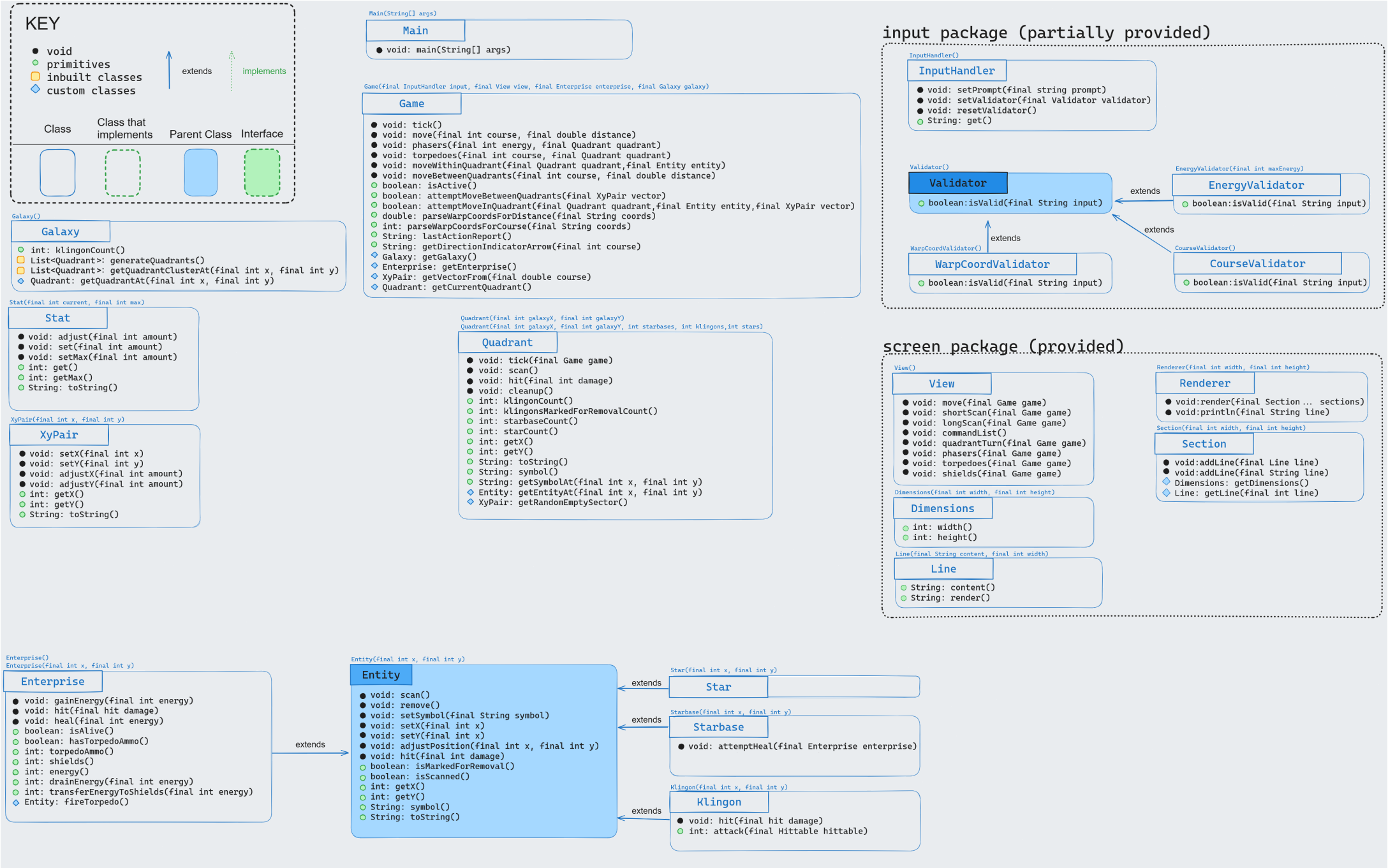

Figure 2: Class diagram of the specification for Individual Assignment 1.

Project Overview

sttrj.game This package contains the core of the game with all the objects and their rules for interaction. Here is the recommended order of proceeding with coding these classes:

- XyPair.java (recommended starting point)
- Stat.java
- Entity.java
- Star.java
- Starbase.java
- Enterprise.java
- Klingon.java
- Quadrant.java
- Galaxy.java
- Game.java

sttrj.input The InputHandler (provided) accepts keyboard input, parses it, and invokes appropriate actions.

- InputHandler.java (will be provided)
- **sttrj.input.validator** (sub-package) A packaged set of input validators, any one of which is used by the InputHandler in validating the users input. The base Validator class (Validator.java) will be provided and accepts all input. You will write the other Validator subclasses:
  - Validator.java (will be provided)
  - EnergyValidator.java
  - CourseValidator.java
  - WarpCoordValidator.java

sttrj.screen This entire package will be provided and implements the user interface.

Grading

Four aspects of your solution will considered in grading your submission. These are *conformance*, *functionality*, *automated style check*, and a manual *human readable style* assessment.

Automated Testing & Checking

Conformance, functionality and automated style check grading will be done automatically in a Linux environment. The environment will not be running Windows, and neither *IntelliJ* nor *Eclipse* (or any other IDE) will be involved. OpenJDK 21 with the JUnit 4 library will be used to compile and execute your code. To prevent infinite loops, or malicious code, from slowing down Gradescope, any test that takes longer than 10 seconds to execute[2] will be killed and identified as failing.

IDEs like *IntelliJ* provide code completion hints. When importing Java libraries they may suggest libraries that are not part of the standard library. These will ***not*** be available in the test environment and your code will ***not*** compile. When uploading your assignment to Gradescope, ***ensure*** that Gradescope says that your submission was compiled successfully.

Your code must compile.
If your submission does not compile, **you will receive no marks**.

Conformance

You are only coding part of this program. Other parts are created by others. Your classes must integrate exactly into the overall program. You must implement the specified classes, and only the specified classes, exactly as described in the Javadocs. Your public methods must have the exact signature specified. (You may add as many private "helper" methods as you see fit, but these must be private and not visible outside their class.) The submission platform will check that your submission matches the expected signatures.

[2] All tests should execute in a small fraction of a second. Any test taking longer than a second to execute indicates faulty logic or malicious code.

Functionality

Each class has a number of unit tests associated with it. Your grade for functionality is based on the percentage of unit tests you pass. Classes may be weighted differently depending on their complexity.

Automated Style Check

Your grade for automated style checking is based on the number of style violations identified by the *Checkstyle* tool[3]. It will be run in the same environment as the JUnit automated functionality tests. Multiple style violations of the same type will ***each*** count as ***additional*** violations.

Note: There is a plug-in available for *IntelliJ* that will highlight style violations in your code. Instructions for installing this plug-in are available in the Java Programming Style Guide on BlackBoard (Learning Resources → Guides). If you correctly use the plug-in and follow the style requirements, it should be relatively straightforward to get high grades for this section.

Human Readable Style

Course staff will grade the *readability* of the code you submit. This will assess the structure, style, documentation and logic of your code. The high-level evaluation is how easily can another programmer, who is familiar with Java, understand your code. This includes layout of code, use of descriptive identifier names, and concise and informative comments. It also includes the detailed design of your code's logic and how much code is unnecessarily duplicated. See Appendix B for criteria that will be used to assess the readability of your code.

Submission

Submission is via Gradescope. Submit your code to Gradescope *early and often*. Gradescope will give you a very small amount feedback on your code every time you submit, including the conformance and automated style tests, but it is *not* a substitute for testing your code yourself. Gradescope will not execute the full functionality assessment before the assignment closes. You are expected to test that your code correctly follows the specification yourself. After the assignment closes (including any extensions and any late submissions), Gradescope will then execute the full functionality assessment and then all results (automated and manual grading marks) will be made available in Gradescope and the raw assignment marks (before any late penalties) will be available in Blackboard. Any late penalties will be applied before final assignment marks are published in Blackboard.

What to Submit Your submission ***must*** have the following internal structure:

`src/`	folders (packages) and `.java` files for classes described in the Javadoc.
`refs.md`	file containing the references for any citations in your code.

Included in the root directory of the provided code are the files `bundle.sh` and `bundle.cmd`. For MacOS and Unix users, run `$bash bundle.sh` file to execute it. For Windows users, double-click the `bundle.cmd` file to execute it. This will create a `submission.zip` file for you to upload to Gradescope.

You can create the submission zip file yourself using a zip utility. If you do this, ***ensure*** that you do not ***miss*** any files or directories. Also ***ensure*** that you do not ***add*** any extra files. We recommend using the provided `bundle` scripts.

Ensure that your classes and interfaces ***correctly*** declare the package they are within. For example, `Enterprise.java` should declare `package sttrj.game;`

Only submit the `src` folder and the `refs.md` file in the root directory of your project.
Do *not* submit ***any*** other files (e.g. no `.class` files or IDE files).

Provided tests A small number of the unit tests (about 10%) used for assessing *functionality* will be provided in Gradescope. These will be used to test your submission, each time you upload it.

These are meant to provide you with an opportunity to receive feedback on whether the basic functionality of your classes works correctly or not. Passing all the provided unit tests does ***not*** guarantee that you will pass all the tests used for functionality grading.

[3] The latest version of the course *Checkstyle* configuration can be found at `http://csse7023.uqcloud.net/checkstyle.xml`. See the *Style Guide* on BlackBoard for instructions on how to use it in *IntelliJ*.

Assessment Policy

Late Submission You must submit your code ***before*** the deadline. Code that is submitted after the deadline will receive a late penalty as described in the course profile. The submission time is determined by the time recorded on the Gradescope server. A submission is not recorded as being received until uploading your files completes. Attempting to submit at the last minute may result in a late submission.

You may submit your assignment to Gradescope as many times as you wish before the due date. If a misconduct case is raised about your submission, a history of regular submissions to Gradescope, which demonstrate progress on your solution, could support your argument that the work was your own.

You are ***strongly*** encouraged to submit your assignment on time, or by your assigned revised deadline if the School of Electrical Engineering and Computer Science has granted you an extension.

Experience has demonstrated that most students who submit their assignments late lose more marks due to the late penalties than they gain by making improvements to their work.

Extensions If an unavoidable disruption occurs (e.g. illness, family crisis, etc.) you should consider applying for an extension. Please refer to the following page for further information.

`https://uq.mu/rl551`

Do not email the course coordinator or other course staff to request an extension. All requests for extensions must be made via my.UQ, ***before*** the submission deadline.

Re-Grading If an *administrative error* has been made in the grading of your assignment, please contact the course coordinator (csse7023@uq.edu.au) to request this be fixed. For all other cases, please refer to the following page for further information.

`https://uq.mu/rl552`

Change Log

Revision: 1.0.0

All changes will be listed in this section of the task sheet.

A Critical Mistakes

THINGS YOU MUST AVOID

This is being heavily emphasised here because these are critical mistakes which **must** be avoided.

Code may run fine locally on your own computer in *IntelliJ*, but it is ***required*** that it also builds and runs correctly when it is executed by the automated grading tool in Gradescope. Your solution needs to conform to the specification for this to occur.

- Files must be in the correct directories ***(exactly)*** as specified by the Javadoc. If files are in incorrect directories *(even **slightly** wrong)*, you may lose grades for functionality in these files because the implementation does not conform to the specification.
- Files must have the correct package declaration at the top of every file. If files have incorrect package declarations *(even **slightly** wrong, such as incorrect capitalisation)*, you may lose grades for functionality in these files because the implementation does not conform to the specification.
- You must implement the public and protected members ***exactly*** as described in the supplied documentation (***no*** *extra public/protected members or classes*). Creating public or protected data members in a class when it is not specified will result in loss of grades, because the implementation does not conform to the specification.
    - You are *encouraged* to create private members as you see fit to implement the required functionality or improve the design of your solution.
- **Do not** import the `org.junit.jupiter.api` package. This is from JUnit 5 and may cause our JUnit tests to fail.
- Do not use ***any*** version of Java other than 21 when writing your solution. If you accidentally use Java features which are different in a version older than 21, then your submission may fail functionality tests. If you accidentally use Java features which are only present in a version newer than 21, then your submission may fail to compile.

© The University of Queensland 2025

B Grading

Automated Testing & Checking

Conformance, functionality, and style checking are assessed via an autograder, automated JUnit tests, and the *Checkstyle* tool.

Your **conformance** marks will be 5% of the total marks, ***LESS*** the number of conformance violations identified. Five or more conformance violations will result in zero marks for this component.

Your **functionality** marks will be 65% of the total marks based on the percentage of unit tests you pass.

Your **automated *Checkstyle*** marks will be 10% of the total marks, ***LESS*** the number of style violations identified. Ten or more style violations will result in zero marks for this component.

Code Style and Structure

The style and structure of your code will be assessed for the remaining 20% of the mark and will be manually assessed by course staff after all of the assignments have been submitted. Style and structure will be graded according to the rubric provided here. The key consideration in grading your code style is whether the code is easy to understand and correctly referenced. Code style will be assessed against the following criteria.

Readability

- Program Structure: Layout of code makes it easier to read and follow its logic. This includes using whitespace to highlight blocks of logic.
- Descriptive Identifier Names: Variable, constant, function, class and method names clearly describe what they represent in the program's logic. Do **not** use what is called the *Hungarian Notation* for identifiers. In short, this means do not include the identifier's type in its name (e.g. `item_list`), rather make the name meaningful. (e.g. Use `items`, where plural informs the reader it is a collection of items and it can easily be changed to be some other collection and not a list.)
- Named Constants: All non-trivial fixed values (literal constants) in the code are represented by descriptive named (symbolic) constants.

Documentation

- Comment Clarity: All significant blocks of code should have a comment to explain how the logic works. For long or complex methods, each logical block should have an in-line comment describing its logic. Comments provide meaningful descriptions of the code. They should not repeat what is already obvious by reading the code (e.g. `# Setting variable to 0.`). Comments should not be verbose or excessive, as this can make it difficult to follow the code.
- Informative Javadoc: Every class, method and member variable4 should have a Javadoc comment that explains its purpose. This includes describing parameters, return values, and potentially thrown exceptions so that others can understand how to use the method correctly.
- Citations and References: You may find ideas of how to solve problems in the assignment through external resources. If you use these ideas you ***must*** cite them. Provide the full bibliographic reference for the resource in file called `refs.md`. The `refs.md` file **must** be in the root folder of your project. Students must clearly reference any use of AI or MT in each instance. A failure to reference generative AI or MT use may constitute student misconduct under the Student Code of Conduct.

Design & Logic

- Single Instance of Logic: Blocks of code should not be duplicated in your program. Any code that needs to be used multiple times should be implemented as a method.
- Variable Scope: Variables should be declared locally in the method in which they are needed. Class variables are avoided, except where they simplify program logic.
- Control Structures: Logic is structured simply and clearly through good use of control structures (e.g. well-designed loops and conditional statements).
- Encapsulation: Classes are designed as self-contained entities with state and behaviour. Methods only directly access the state of the object on which they were invoked. Methods never update the state of another object.

© The University of Queensland 2025